Northern University of Bussiness and Technology Khulna

MIT $(^w^)$

Sourav Mondal, Rahul Kumar Ghosh, Ayon Adikary Arko

adapted from KACTL and MIT NULL

2025-10-15

1 Contest 1

2 Data Structures 1

Contest (1)

TemplateShort.cpp

77aa31, 44 lines

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using db = long double; // or double if tight TL
using str = string;

using pi = pair<int,int>;
#define mp make_pair
#define f first
#define s second

#define tcT template<class T
tcT> using V = vector<T>;
tcT, size_t SZ> using AR = array<T,SZ>;
using vi = V<int>;
using vb = V<bool>;
using vpi = V<pi>;

#define sz(x) int((x).size())
#define all(x) begin(x), end(x)
#define sor(x) sort(all(x))
#define rsz resize
#define pb push_back
#define ft front()
#define bk back()

#define FOR(i,a,b) for (int i = (a); i < (b); ++i)
#define F0R(i,a) FOR(i,0,a)
#define ROF(i,a,b) for (int i = (b)-1; i >= (a); --i)
#define R0F(i,a) ROF(i,0,a)
#define rep(a) F0R(_,a)
#define each(a,x) for (auto& a: x)

const int MOD = 1e9+7;
const db PI = acos((db)-1);
mt19937 rng(0); // or mt19937_64

tcT> bool ckmin(T& a, const T& b) {
  return b < a ? a = b, 1 : 0; } // set a = min(a,b)
tcT> bool ckmax(T& a, const T& b) {
  return a < b ? a = b, 1 : 0; } // set a = max(a,b)

int main() { cin.tie(0)->sync_with_stdio(0); }
```

.bashrc

3 lines

```
alias clr="printf '\33c'"
co() { g++ -std=c++17 -O2 -Wall -Wextra -Wshadow -Wconversion -
    ↪o $1 $1.cpp; }
run() { co $1 && ./$1; }
```

hash.sh

3 lines

```
# Hash file ignoring whitespace and comments. Verifies that
# code was correctly typed. Usage: 'sh hash.sh < A.cpp'
cpp -dD -P -fpreprocessed|tr -d '[:space:]'|md5sum|cut -c-6
```

stress.sh

10 lines

```
# A and B are executables you want to compare, gen takes int
# as command line arg. Usage: 'sh stress.sh'
for((i = 1; ; ++i)); do
    echo $i
    ./gen $i > int
    ./A < int > out1
    ./B < int > out2
    diff -w out1 out2 || break
    # diff -w <(./A < int) <(./B < int) || break
done
```

troubleshoot.txt

75 lines

```
General:
Write down most of your thoughts, even if you're not sure
whether they're useful.
Give your variables (and files) meaningful names.
Stay organized and don't leave papers all over the place!
You should know what your code is doing ...

Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Remove debug output.
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output as well.
Read the full problem statement again.
Have you understood the problem correctly?
Are you sure your algorithm works?
Try writing a slow (but correct) solution.
Can your algorithm handle the whole range of input?
Did you consider corner cases (ex. n=1)?
Is your output format correct? (including whitespace)
Are you clearing all data structures between test cases?
Any uninitialized variables?
Any undefined behavior (array out of bounds)?
Any overflows or NaNs (or shifting ll by >=64 bits)?
Confusing N and M, i and j, etc.?
Confusing ++i and i++?
Return vs continue vs break?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some test cases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Rewrite your solution from the start or let a teammate do it.

Geometry:
Work with ints if possible.
Correctly account for numbers close to (but not) zero. Related:
for functions like acos make sure absolute val of input is not
(slightly) greater than one.
Correctly deal with vertices that are collinear, concyclic,
coplanar (in 3D), etc.
Subtracting a point from every other (but not itself)?

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What's your complexity? Large TL does not mean that something
simple (like NlogN) isn't intended.
Are you copying a lot of unnecessary data? (References)
Avoid vector, map. (use arrays/unordered_map)
How big is the input and output? (consider FastIO)
What do your teammates think about your algorithm?
Calling count() on multiset?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
If using pointers try BumpAllocator.
```

Data Structures (2)

2.1 STL

MapComparator.h

Description: example of function object (functor) for map or set
Usage: `set<int,cmp> s; map<int,int,cmp> m;`

5bfa6c, 1 lines

```
struct cmp{bool operator()(int l,int r)const{return l>r;}};
```

HashMap.h

Description: Hash map with similar API as unordered_map. Initial capacity must be a power of 2 if provided.
Usage: `ht<int,int> h({},{},{},{},{1<<16});`
Memory: ~1.5x unordered map
Time: ~3x faster than unordered map

<ext/pb_ds/assoc_container.hpp> 5872b2, 9 lines

```
using namespace __gnu_pbds;
struct chash {
  const uint64_t C = ll(4e18*acos(0))+71; // large odd number
  const int RANDOM = rng();
  ll operator()(ll x) const { return __builtin_bswap64((x^
    ↪RANDOM)*C); }
};
template<class K,class V> using ht = gp_hash_table<K,V,chash>;
template<class K,class V> V get(ht<K,V>& u, K x) {
  auto it = u.find(x); return it == end(u) ? 0 : it->s; }
```

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. Change `null_type` to get a map.
Time: $\mathcal{O}(\log N)$

<ext/pb_ds/assoc_container.hpp> fe2351, 12 lines

```
using namespace __gnu_pbds;
tcT> using Tree = tree<T, null_type, less<T>,
  rb_tree_tag, tree_order_statistics_node_update>;
#define ook order_of_key
#define fbo find_by_order

void treeExample() {
  Tree<int> t, t2; t.insert(8);
  auto it = t.insert(10).f; assert(it == t.lb(9));
  assert(t.ook(10) == 1 && t.ook(11) == 2 && *t.fbo(0) == 8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

LineContainer.h

Description: Add lines of the form $ax + b$, query maximum y-coordinate for any x.
Time: $\mathcal{O}(\log N)$

df7386, 29 lines

```
using T = ll; const T INF = LLONG_MAX; // a/b rounded down
// ll fdiv(ll a, ll b) { return a/b-((a^b)<0&&a%b); }

bool _Q = 0;
struct Line {
  T a, b; mutable T lst;
  T eval(T x) const { return a*x+b; }
  bool operator<(const Line&o)const{return _Q?lst<o.lst:a<o.a;}
  T last_gre(const Line& o) const { assert(a <= o.a);
    // greatest x s.t. a*x+b >= o.a*x+o.b
    return lst=(a==o.a?(b>=o.b?INF:-INF):fdiv(b-o.b,o.a-a));}
};

struct LineContainer: multiset<Line> {
  bool isect(iterator it) { auto n_it = next(it);
    if (n_it == end()) return it->lst = INF, 0;
    return it->last_gre(*n_it) >= n_it->lst; }
  void add(T a, T b) {
    auto it = ins({a,b,0}); while (isect(it)) erase(next(it));
    if (it == begin()) return;
    if (isect(--it)) erase(next(it)), isect(it);
    while (it != begin()) {
      --it; if (it->lst < next(it)->lst) break;
      erase(next(it)); isect(it); }
  }
  T qmax(T x) { assert(!empty());
    _Q = 1; T res = lb({0,0,x})->eval(x); _Q = 0;
    return res; }
};
```

2.2 1D Range Queries

RMQ.h

Description: 1D range minimum query. If TL is an issue, use arrays instead of vectors and store values instead of indices.
Memory: $\mathcal{O}(N \log N)$
Time: $\mathcal{O}(1)$

a3f881, 19 lines

```
tcT> struct RMQ { // floor(log_2(x))
  int level(int x) { return 31-__builtin_clz(x); }
  V<T> v; V<vi> jmp;
  int cmb(int a, int b) {
    return v[a]==v[b]?min(a,b):(v[a]<v[b]?a:b); }
  void init(const V<T>& _v) {
    v = _v; jmp = {vi(sz(v))};
    iota(all(jmp[0]),0);
    for (int j = 1; 1<<j <= sz(v); ++j) {
      jmp.pb(vi(sz(v)-(1<<j)+1));
      F0R(i,sz(jmp[j])) jmp[j][i] = cmb(jmp[j-1][i],
        jmp[j-1][i+(1<<(j-1))]);
    }
  }
  int index(int l, int r) {
    assert(l <= r); int d = level(r-l+1);
    return cmb(jmp[d][l],jmp[d][r-(1<<d)+1]); }
  T query(int l, int r) { return v[index(l,r)]; }
};
```

SegmentTree.h

Description: 1D point update and range query where `cmb` is any associative operation. `seg[1]==query(0,N-1)`.
Time: $\mathcal{O}(\log N)$

1630f3, 18 lines

```
tcT> struct SegTree { // cmb(ID,b) = b
  const T ID{}; T cmb(T a, T b) { return a+b; }
  int n; V<T> seg;
  void init(int _n) { // upd, query also work if n = _n
    for (n = 1; n < _n; ) n *= 2;
    seg.assign(2*n,ID); }
  void pull(int p) { seg[p] = cmb(seg[2*p],seg[2*p+1]); }
  void upd(int p, T val) { // set val at position p
    seg[p += n] = val; for (p /= 2; p; p /= 2) pull(p); }
  T query(int l, int r) { // zero-indexed, inclusive
    T ra = ID, rb = ID;
    for (l += n, r += n+1; l < r; l /= 2, r /= 2) {
      if (l&1) ra = cmb(ra,seg[l++]);
      if (r&1) rb = cmb(seg[--r],rb);
    }
    return cmb(ra,rb);
  }
};
```

LazySegmentTree.h

Description: 1D range increment and sum query.
Time: $\mathcal{O}(\log N)$

78a06d, 26 lines

```
tcT, int SZ> struct LazySeg {
  static_assert(pct(SZ) == 1); // SZ must be power of 2
  const T ID{}; T cmb(T a, T b) { return a+b; }
  T seg[2*SZ], lazy[2*SZ];
  LazySeg() { F0R(i,2*SZ) seg[i] = lazy[i] = ID; }
  void push(int ind, int L, int R) {
    seg[ind] += (R-L+1)*lazy[ind]; // dependent on operation
    if (L != R) F0R(i,2) lazy[2*ind+i] += lazy[ind];
    lazy[ind] = 0;
  } // recalc values for current node
  void pull(int ind){seg[ind]=cmb(seg[2*ind],seg[2*ind+1]);}
  void build() { ROF(i,1,SZ) pull(i); }
  void upd(int lo,int hi,T inc,int ind=1,int L=0, int R=SZ-1) {
    push(ind,L,R); if (hi < L || R < lo) return;
    if (lo <= L && R <= hi) {
      lazy[ind] = inc; push(ind,L,R); return; }
    int M = (L+R)/2; upd(lo,hi,inc,2*ind,L,M);
    upd(lo,hi,inc,2*ind+1,M+1,R); pull(ind);
  }
  T query(int lo, int hi, int ind=1, int L=0, int R=SZ-1) {
    push(ind,L,R); if (lo > R || L > hi) return ID;
    if (lo <= L && R <= hi) return seg[ind];
    int M = (L+R)/2; return cmb(query(lo,hi,2*ind,L,M),
      query(lo,hi,2*ind+1,M+1,R));
  }
};
```

PSeg.h

Description: Persistent min segtree with lazy updates, no propagation. If making `d` a vector then save the results of `upd` and `build` in local variables first to avoid issues when vector resizes in C++14 or lower.
Memory: $\mathcal{O}(N + Q \log N)$

8f37fa, 46 lines

```
tcT, int SZ> struct pseg {
  static const int LIM = 2e7;
  struct node {
    int l, r; T val = 0, lazy = 0;
    void inc(T x) { lazy += x; }
    T get() { return val+lazy; }
  };
  node d[LIM]; int nex = 0;
  int copy(int c) { d[nex] = d[c]; return nex++; }
  T cmb(T a, T b) { return min(a,b); }
  void pull(int c) { d[c].val =
    cmb(d[d[c].l].get(), d[d[c].r].get()); }
  //// MAIN FUNCTIONS
  T query(int c, int lo, int hi, int L, int R) {
    if (lo <= L && R <= hi) return d[c].get();
    if (R < lo || hi < L) return MOD;
    int M = (L+R)/2;
    return d[c].lazy+cmb(query(d[c].l,lo,hi,L,M),
              query(d[c].r,lo,hi,M+1,R));
  }
  int upd(int c, int lo, int hi, T v, int L, int R) {
    if (R < lo || hi < L) return c;
    int x = copy(c);
    if (lo <= L && R <= hi) { d[x].inc(v); return x; }
    int M = (L+R)/2;
    d[x].l = upd(d[x].l,lo,hi,v,L,M);
    d[x].r = upd(d[x].r,lo,hi,v,M+1,R);
    pull(x); return x;
  }
  int build(const V<T>& arr, int L, int R) {
    int c = nex++;
    if (L == R) {
      if (L < sz(arr)) d[c].val = arr[L];
      return c;
    }
    int M = (L+R)/2;
    d[c].l = build(arr,L,M), d[c].r = build(arr,M+1,R);
    pull(c); return c;
  }
  vi loc; //// PUBLIC
  void upd(int lo, int hi, T v) {
    loc.pb(upd(loc.bk,lo,hi,v,0,SZ-1)); }
  T query(int ti, int lo, int hi) {
    return query(loc[ti],lo,hi,0,SZ-1); }
  void build(const V<T>&arr) {loc.pb(build(arr,0,SZ-1));}
};
```

Treap.h

Description: Easy BBST. Use split and merge to implement insert and delete.
Time: $\mathcal{O}(\log N)$

bdb758, 65 lines

```
using pt = struct tnode*;
struct tnode {
  int pri, val; pt c[2]; // essential
  int sz; ll sum; // for range queries
  bool flip = 0; // lazy update
  tnode(int _val) {
    pri = rng(); sum = val = _val;
    sz = 1; c[0] = c[1] = nullptr;
  }
  ~tnode() { F0R(i,2) delete c[i]; }
};
int getsz(pt x) { return x?x->sz:0; }
ll getsum(pt x) { return x?x->sum:0; }
pt prop(pt x) { // lazy propagation
  if (!x || !x->flip) return x;
  swap(x->c[0],x->c[1]);
  x->flip = 0; F0R(i,2) if (x->c[i]) x->c[i]->flip ^= 1;
  return x;
}
pt calc(pt x) {
  pt a = x->c[0], b = x->c[1];
  assert(!x->flip); prop(a), prop(b);
  x->sz = 1+getsz(a)+getsz(b);
  x->sum = x->val+getsum(a)+getsum(b);
  return x;
}
void tour(pt x, vi& v) { // print values of nodes,
  if (!x) return; // inorder traversal
  prop(x); tour(x->c[0],v); v.pb(x->val); tour(x->c[1],v);
}
pair<pt,pt> split(pt t, int v) { // >= v goes to the right
  if (!t) return {t,t};
```

```
  prop(t);
  if (t->val >= v) {
    auto p = split(t->c[0], v); t->c[0] = p.s;
    return {p.f,calc(t)};
  } else {
    auto p = split(t->c[1], v); t->c[1] = p.f;
    return {calc(t),p.s};
  }
}
pair<pt,pt> splitsz(pt t, int sz) { // sz nodes go to left
  if (!t) return {t,t};
  prop(t);
  if (getsz(t->c[0]) >= sz) {
    auto p = splitsz(t->c[0],sz); t->c[0] = p.s;
    return {p.f,calc(t)};
  } else {
    auto p=splitsz(t->c[1],sz-getsz(t->c[0])-1); t->c[1]=p.f;
    return {calc(t),p.s};
  }
}
pt merge(pt l, pt r) { //  keys in l < keys in r
  if (!l || !r) return l?:r;
  prop(l), prop(r); pt t;
  if (l->pri > r->pri) l->c[1] = merge(l->c[1],r), t = l;
  else r->c[0] = merge(l,r->c[0]), t = r;
  return calc(t);
}
pt ins(pt x, int v) { // insert v
  auto a = split(x,v), b = split(a.s,v+1);
  return merge(a.f,merge(new tnode(v),b.s)); }
pt del(pt x, int v) { // delete v
  auto a = split(x,v), b = split(a.s,v+1);
  return merge(a.f,b.s); }
```